Compression & Quantization

Question: If a system has 100 million user-product interactions, and if each order is mapped to a vector of 768 dimensions, how much space will the system require to store all vectors?

Answer: 100 million interactions = 10^8 interactions.
Each order is stored as a 768-dimensional vector. That's 7.68×10^{10} values.

If every dimensional value is stored as a 32-bit floating point, we need $7.68 \times 10^{10} \times 32$ bits.

$= 7.68 \times 10^{10} \times 4$ Bytes

$= 3.072 \times 10^{11}$ Bytes

= 307.2 GB

Caching this data is very challenging.

This makes the system slow to query, hurting user experience.

Why Compress

The goal of compression is not saving disk space. Compression helps:

- Fitting **more vectors in RAM**
- Reducing **search latency**
- Making **SIMD or GPU** compute feasible

How It Works

1. Product Quantization (PQ)

- Break each vector into **sub-vectors** (e.g., split a 128D vector into 8 chunks of 16D).
- For each chunk, find the **nearest centroid** from a pre-trained codebook.

- Store only the **index** of the centroid, not the float values.

So instead of storing 128 floats (512 bytes), you store 8 integers (8 bytes).
That's a **64x reduction**.

PQ is used heavily in Facebook's FAISS, Milvus, and other modern vector DBs.

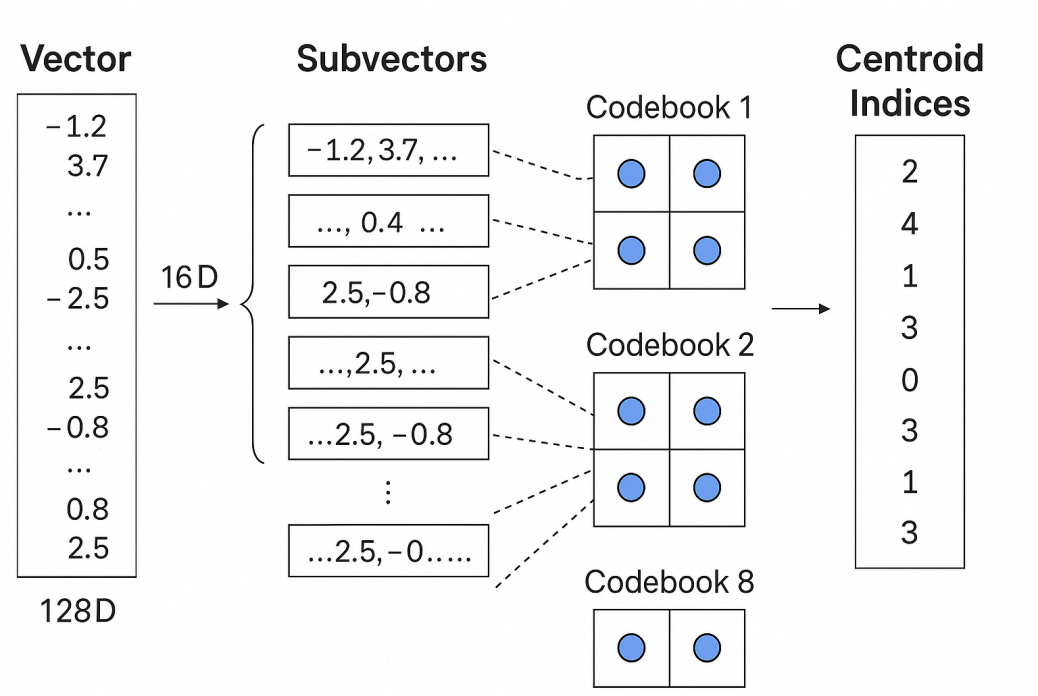

Figure 1: Product Quantization maps vector chunks to codes, reducing dimensionality and size.

---

2. Scalar Quantization (SQ)

- Compress each float in the vector individually.
- Convert from 32-bit float to 8-bit (or less) integer using fixed scale and offset.

This is simpler than PQ but less precise. Often combined with vector normalization.

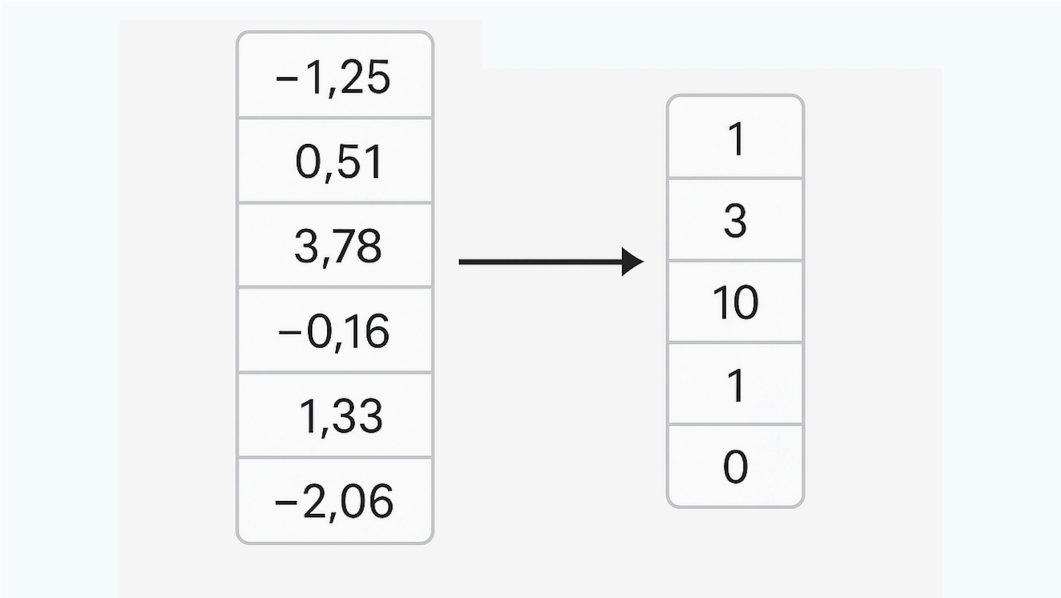

Figure 2: Scalar Quantization compresses raw values to lower-bit representations.

Trade-off

Quantization saves lots of RAM and gives a big speed boost.

But search precision and recall drops if you're too aggressive (e.g., 4-bit PQ makes the vectors too "blurry").

Compression artifacts also hurt the ***long tail of similarity***: close-but-not-close-enough neighbors.

1. Original high-dimensional vectors → accurately capture fine-grained relationships.
2. After compression very close neighbors will still match correctly. But slightly farther yet relevant vectors get perturbed just enough to drop out of the result list.
3. These errors don’t affect top-1 accuracy much but kill recall in the tail.

Imagine you’re tuning a radio:

- Strong stations (top matches) still come through fine.
- Weak but still interesting stations (long tail) get lost in static due to compression noise.

Impact:

- ***Bad for recommendations***, where diversity and recall matter.
- ***Harms ranking*** in search results.

Key Takeaways

- Vector compression allows fast, scalable search.
- PQ: Sub-vector + codebook trick (most powerful).
- SQ: Per-float quantization.
- INT8: Hardware-friendly, model-compatible.
- Always balance: size vs recall vs latency.